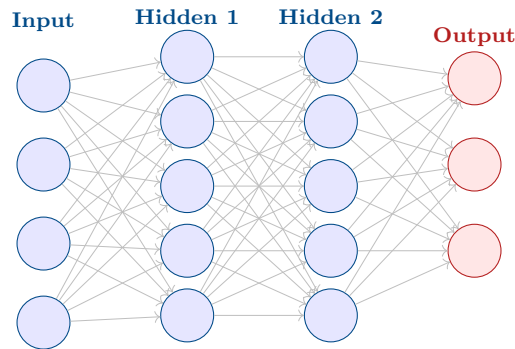


Advanced Machine Learning

MSc 2-Credit Course — Complete Study Notes



Topics Covered:

Supervised & Unsupervised Learning · Multi-Task Learning · Bayesian Modelling · Gaussian Processes · Randomized Methods · Bayesian Neural Networks · Approximate Inference · Variational Autoencoders · Generative Models · RNN / BPTT / LSTM · Attention & Memory Networks · Neural Turing Machines · Machine Translation · Question Answering · Speech Recognition · Syntactic & Semantic Parsing · GPU Optimisation

Department of Computer Science

MSc in Machine Learning & AI

March 31, 2026

Contents

I	Core Learning Paradigms	3
1	Supervised, Unsupervised, and Multi-Task Learning	4
1.1	Supervised Learning	4
1.1.1	Empirical Risk Minimisation (ERM)	4
1.1.2	Key Algorithms	4
1.2	Unsupervised Learning	4
1.2.1	Clustering	5
1.2.2	Dimensionality Reduction	5
1.3	Multi-Task Learning	5
1.3.1	Hard vs. Soft Parameter Sharing	5
II	Bayesian and Probabilistic Methods	6
2	Bayesian Modelling and Gaussian Processes	7
2.1	Bayesian Inference Fundamentals	7
2.1.1	Predictive Distribution	7
2.2	Gaussian Processes	7
2.2.1	GP Regression	8
2.2.2	Kernel Functions	8
2.3	Randomized Methods in Machine Learning	8
2.3.1	Random Projections (Johnson–Lindenstrauss)	8
2.3.2	Random Features for Kernel Approximation	8
2.3.3	Stochastic Gradient Descent (SGD)	9
3	Bayesian Neural Networks and Approximate Inference	10
3.1	Bayesian Neural Networks (BNNs)	10
3.1.1	Benefits and Challenges	10
3.2	Approximate Inference	10
3.2.1	Markov Chain Monte Carlo (MCMC)	10
3.2.2	Variational Inference (VI)	11
3.2.3	Monte Carlo Dropout	11
4	Variational Autoencoders and Generative Models	12
4.1	Variational Autoencoders (VAEs)	12
4.1.1	The ELBO Objective	12

4.1.2	Reparameterisation Trick	12
4.2	Generative Models	12
4.2.1	Generative Adversarial Networks (GANs)	12
4.2.2	Normalising Flows	12
4.2.3	Diffusion Models	13
III	NLP and Deep Sequence Models	14
5	Recurrent Neural Networks	15
5.1	Vanilla RNN	15
5.2	Backpropagation Through Time (BPTT)	15
5.2.1	Vanishing and Exploding Gradients	15
5.3	Long Short-Term Memory (LSTM)	16
6	Attention, Memory, and Neural Turing Machines	17
6.1	Attention Mechanisms	17
6.1.1	Multi-Head Attention (Transformer)	17
6.2	Memory Networks	18
6.3	Neural Turing Machines (NTMs)	18
7	NLP Applications	19
7.1	Machine Translation	19
7.1.1	Encoder–Decoder Architecture	19
7.1.2	Transformer for Translation	19
7.2	Question Answering (QA)	19
7.3	Speech Recognition	19
7.3.1	Connectionist Temporal Classification (CTC)	20
7.3.2	Attention-Based Encoder–Decoder (AED)	20
7.3.3	Whisper / wav2vec 2.0	20
7.4	Syntactic and Semantic Parsing	20
7.4.1	Syntactic Parsing	20
7.4.2	Semantic Parsing	20
8	GPU Optimisation for Neural Networks	21
8.1	Hardware Landscape	21
8.2	Memory Optimisation	21
8.3	Compute Optimisation	21
8.4	Distributed Training at Scale	21
	Key References	23

Part I

Core Learning Paradigms

Chapter 1

Supervised, Unsupervised, and Multi-Task Learning

1.1 Supervised Learning

Definition 1.1.1: Supervised Learning

Given a labelled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, supervised learning finds a function $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ that minimises a loss $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(\mathbf{x}_i), y_i)$.

1.1.1 Empirical Risk Minimisation (ERM)

The goal is to minimise the expected risk:

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}}[\ell(f(\mathbf{x}), y)] \approx \hat{R}(f) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i), y_i).$$

Regularisation (L1/L2) controls overfitting:

$$\mathcal{L}_{\text{reg}}(\theta) = \hat{R}(f) + \lambda \|\theta\|_p^p, \quad p \in \{1, 2\}.$$

1.1.2 Key Algorithms

- **Linear & Logistic Regression** — closed-form or gradient descent.
- **Support Vector Machines** — maximise margin; kernel trick for non-linearity.
- **Decision Trees / Random Forests / Gradient Boosting** — ensemble methods.
- **Deep Neural Networks** — universal function approximators trained via SGD.

1.2 Unsupervised Learning

Definition 1.2.1: Unsupervised Learning

Learning structure from unlabelled data $\{\mathbf{x}_i\}_{i=1}^N$ by modelling the data distribution $p(\mathbf{x})$ or finding compact representations.

1.2.1 Clustering

K-Means minimises the within-cluster sum of squares:

$$\min_{\{\mu_k\}} \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mu_k\|^2.$$

Gaussian Mixture Models (GMM) provide a probabilistic generalisation:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k), \quad \sum_k \pi_k = 1.$$

Parameters are estimated via the *Expectation-Maximisation (EM)* algorithm.

1.2.2 Dimensionality Reduction

Principal Component Analysis (PCA) projects data onto the top- d eigenvectors of the covariance matrix $\Sigma = \frac{1}{N} \mathbf{X}^\top \mathbf{X}$. **Autoencoders** learn a low-dimensional code $\mathbf{z} = \text{enc}_\phi(\mathbf{x})$ and reconstruction $\hat{\mathbf{x}} = \text{dec}_\theta(\mathbf{z})$.

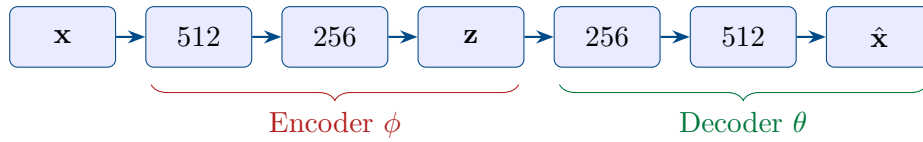


Figure 1.1: Standard Autoencoder architecture: encoder compresses $\mathbf{x}$ to latent code $\mathbf{z}$; decoder reconstructs $\hat{\mathbf{x}}$.

1.3 Multi-Task Learning

Definition 1.3.1: Multi-Task Learning (MTL)

MTL trains a model jointly on T related tasks, sharing representations to improve generalisation. The objective is:

$$\min_{\theta} \sum_{t=1}^T w_t \mathcal{L}_t(\theta_{\text{shared}}, \theta_t)$$

where θ_{shared} is a common backbone and θ_t are task-specific heads.

1.3.1 Hard vs. Soft Parameter Sharing

- **Hard sharing:** All tasks share hidden layers; only output layers differ.
- **Soft sharing:** Each task has its own network; parameters are regularised to stay close (e.g. $\|\theta_t - \theta_{t'}\|^2 \leq \epsilon$).
- **Cross-stitch networks:** Learn linear combinations of activations across tasks at each layer.

Part II

Bayesian and Probabilistic Methods

Chapter 2

Bayesian Modelling and Gaussian Processes

2.1 Bayesian Inference Fundamentals

Definition 2.1.1: Bayes' Theorem

$$\underbrace{p(\theta \mid \mathcal{D})}_{\text{posterior}} = \frac{\overbrace{p(\mathcal{D} \mid \theta)}^{\text{likelihood}} \overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{marginal likelihood}}}$$

The posterior captures our updated belief about parameters θ after observing data $\mathcal{D}$.

2.1.1 Predictive Distribution

Rather than a point estimate, Bayesian methods give:

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) = \int p(y^* \mid \mathbf{x}^*, \theta) p(\theta \mid \mathcal{D}) d\theta.$$

This integral is generally intractable and motivates approximate inference.

2.2 Gaussian Processes

Definition 2.2.1: Gaussian Process

A Gaussian Process (GP) is a collection of random variables, any finite number of which are jointly Gaussian. It is fully specified by a mean function $m(\mathbf{x})$ and a covariance (kernel) function $k(\mathbf{x}, \mathbf{x}')$:

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

2.2.1 GP Regression

Given observations $\mathbf{y} = f(\mathbf{X}) + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_n^2 \mathbf{I})$, the posterior predictive at test points $\mathbf{X}_*$ is:

$$\mu_* = \mathbf{K}_{*\mathbf{X}} (\mathbf{K}_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, \quad (2.1)$$

$$\Sigma_* = \mathbf{K}_{**} - \mathbf{K}_{*\mathbf{X}} (\mathbf{K}_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{K}_{\mathbf{X}*}, \quad (2.2)$$

where $[\mathbf{K}_{\mathbf{X}\mathbf{X}}]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

2.2.2 Kernel Functions

Kernel	Formula	Property
Squared Exponential (RBF)	$\exp\left(-\frac{\ \mathbf{x}-\mathbf{x}'\ ^2}{2\ell^2}\right)$	Infinitely differentiable
Matérn 5/2	$\left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) e^{-\frac{\sqrt{5}r}{\ell}}$	Twice differentiable
Periodic	$\exp\left(-\frac{2\sin^2(\pi x-x' /p)}{\ell^2}\right)$	Captures periodicity
Linear	$\sigma_b^2 + \sigma_v^2(\mathbf{x} \cdot \mathbf{x}')$	Bayesian linear regression

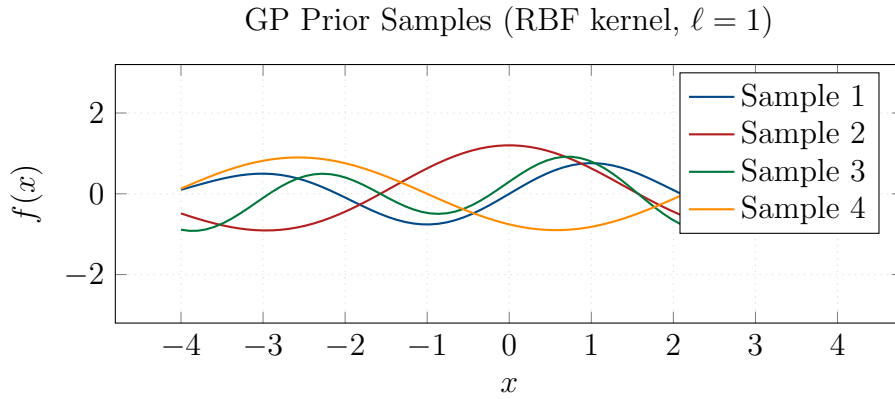


Figure 2.1: Samples from a GP prior with RBF kernel and shaded $\pm 2\sigma$ uncertainty band.

2.3 Randomized Methods in Machine Learning

2.3.1 Random Projections (Johnson–Lindenstrauss)

Theorem 2.3.1: Johnson–Lindenstrauss Lemma

For any $0 < \epsilon < 1$ and n points in $\mathbb{R}^d$, a random projection $\mathbf{A} \in \mathbb{R}^{k \times d}$ with $k = O(\log n / \epsilon^2)$ satisfies $(1 - \epsilon)\|\mathbf{u} - \mathbf{v}\|^2 \leq \|\mathbf{A}\mathbf{u} - \mathbf{A}\mathbf{v}\|^2 \leq (1 + \epsilon)\|\mathbf{u} - \mathbf{v}\|^2$ with high probability.

2.3.2 Random Features for Kernel Approximation

Bochner's theorem: any shift-invariant kernel can be written $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{\omega}[e^{j\omega^\top(\mathbf{x}-\mathbf{x}')}]$. Approximate with D random Fourier features:

$$k(\mathbf{x}, \mathbf{x}') \approx \phi(\mathbf{x})^\top \phi(\mathbf{x}'), \quad \phi(\mathbf{x}) = \frac{1}{\sqrt{D}}[\cos(\omega_i^\top \mathbf{x} + b_i)]_{i=1}^D,$$

reducing kernel machine complexity from $O(N^2)$ to $O(ND)$.

2.3.3 Stochastic Gradient Descent (SGD)

Mini-batch SGD approximates the gradient using a batch $\mathcal{B} \subset [N]$:

$$\theta \leftarrow \theta - \eta \frac{N}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell_i(\theta).$$

Variants include Adam, AdaGrad, RMSProp, each adapting per-parameter learning rates.

Chapter 3

Bayesian Neural Networks and Approximate Inference

3.1 Bayesian Neural Networks (BNNs)

Definition 3.1.1: Bayesian Neural Network

A BNN places a prior distribution $p(\mathbf{W})$ over network weights instead of treating them as point estimates. The posterior after observing data is:

$$p(\mathbf{W} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \mathbf{W}) p(\mathbf{W})}{p(\mathcal{D})}.$$

Predictions are made by *Bayesian model averaging*:

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) = \int p(y^* \mid \mathbf{x}^*, \mathbf{W}) p(\mathbf{W} \mid \mathcal{D}) d\mathbf{W}.$$

3.1.1 Benefits and Challenges

- **Benefits:** Principled uncertainty quantification; resistance to overfitting; natural regularisation via priors.
- **Challenges:** The posterior $p(\mathbf{W} \mid \mathcal{D})$ is intractable for modern networks; requires approximate inference.

3.2 Approximate Inference

3.2.1 Markov Chain Monte Carlo (MCMC)

MCMC constructs a Markov chain whose stationary distribution is the target posterior. **Hamiltonian Monte Carlo (HMC)** uses gradient information to propose distant samples with high acceptance probability:

$$H(\mathbf{q}, \mathbf{p}) = U(\mathbf{q}) + K(\mathbf{p}), \quad U(\mathbf{q}) = -\log p(\mathbf{q} \mid \mathcal{D}), \quad K(\mathbf{p}) = \frac{1}{2} \mathbf{p}^\top M^{-1} \mathbf{p}.$$

3.2.2 Variational Inference (VI)

VI approximates the intractable posterior $p(\theta \mid \mathcal{D})$ by optimising a simpler distribution $q_\phi(\theta)$ to minimise the KL divergence:

$$q_\phi^* = \arg \min_{q \in \mathcal{Q}} \text{KL}[q_\phi(\theta) \parallel p(\theta \mid \mathcal{D})].$$

This is equivalent to maximising the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}(\phi) = \underbrace{\mathbb{E}_{q_\phi}[\log p(\mathcal{D} \mid \theta)]}_{\text{expected log-likelihood}} - \underbrace{\text{KL}[q_\phi(\theta) \parallel p(\theta)]}_{\text{regularisation}}.$$

3.2.3 Monte Carlo Dropout

Dropout at test time (Gal & Ghahramani, 2016) provides a cheap BNN approximation: run T stochastic forward passes and use sample mean/variance as predictive mean/uncertainty:

$$\hat{y} \approx \frac{1}{T} \sum_{t=1}^T f_{\hat{\mathbf{W}}_t}(\mathbf{x}^*), \quad \hat{\mathbf{W}}_t \sim q_\phi(\mathbf{W}).$$

Chapter 4

Variational Autoencoders and Generative Models

4.1 Variational Autoencoders (VAEs)

Definition 4.1.1: VAE

A VAE is a latent-variable generative model $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$ where $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and the likelihood $p_\theta(\mathbf{x} | \mathbf{z})$ is a neural decoder. Inference uses an amortised encoder $q_\phi(\mathbf{z} | \mathbf{x})$.

4.1.1 The ELBO Objective

The VAE maximises:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} [\log p_\theta(\mathbf{x} | \mathbf{z})]}_{\text{reconstruction}} - \underbrace{\text{KL}[q_\phi(\mathbf{z} | \mathbf{x}) || p(\mathbf{z})]}_{\text{regularisation}}. \quad (4.1)$$

4.1.2 Reparameterisation Trick

The key trick enabling backpropagation through stochastic nodes:

$$\mathbf{z} = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

4.2 Generative Models

4.2.1 Generative Adversarial Networks (GANs)

A GAN comprises a generator G and discriminator D playing a minimax game:

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

4.2.2 Normalising Flows

Learn an invertible mapping $f : \mathcal{Z} \rightarrow \mathcal{X}$; exact log-likelihood via the change-of-variables formula:

$$\log p(\mathbf{x}) = \log p_{\mathbf{z}}(f^{-1}(\mathbf{x})) + \log \left| \det \frac{\partial f^{-1}}{\partial \mathbf{x}} \right|.$$

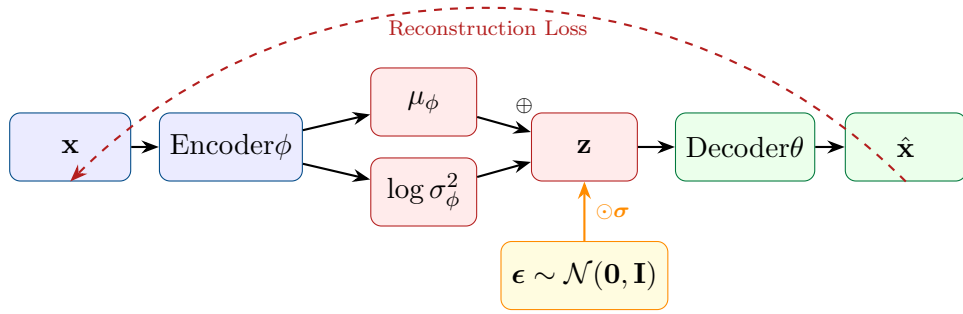


Figure 4.1: VAE architecture showing the encoder, reparameterisation trick, decoder, and training signal flow.

4.2.3 Diffusion Models

Forward process adds Gaussian noise over T steps: $q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$. A neural network learns the reverse denoising process $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$.

Model	Exact LL	Sample Quality	Training Stability
VAE	lower bound	moderate	stable
GAN	no	high	unstable (mode collapse)
Flow	yes	moderate	stable
Diffusion	lower bound	very high	stable

Part III

NLP and Deep Sequence Models

Chapter 5

Recurrent Neural Networks

5.1 Vanilla RNN

Definition 5.1.1: Recurrent Neural Network

An RNN maintains a hidden state $\mathbf{h}_t$ updated at each time step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = \mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y.$$

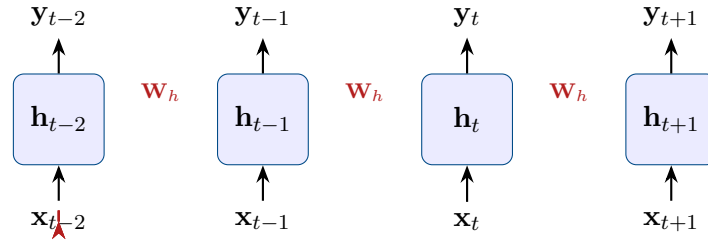


Figure 5.1: Unrolled RNN across four time steps; $\mathbf{W}_h$ is shared.

5.2 Backpropagation Through Time (BPTT)

Gradients are computed by unrolling the network and applying the chain rule. The gradient of loss $\mathcal{L}$ w.r.t. $\mathbf{W}_h$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \sum_{k=1}^t \left(\prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_h}.$$

5.2.1 Vanishing and Exploding Gradients

The Jacobian product $\prod_j \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ causes gradients to either vanish ($\rightarrow 0$) or explode ($\rightarrow \infty$) exponentially in the sequence length. Gradient clipping addresses explosion: $\hat{g} \leftarrow \min\left(1, \frac{\tau}{\|g\|}\right) g$.

5.3 Long Short-Term Memory (LSTM)

The LSTM (Hochreiter & Schmidhuber, 1997) introduces a *cell state* $\mathbf{c}_t$ with gating mechanisms to carry information over long sequences.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate}) \quad (5.1)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate}) \quad (5.2)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{candidate cell}) \quad (5.3)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (\text{cell update}) \quad (5.4)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate}) \quad (5.5)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{hidden state}) \quad (5.6)$$

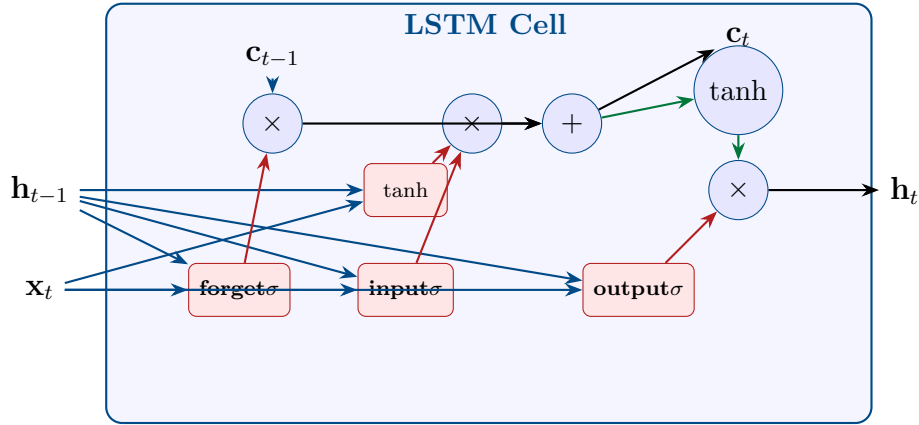


Figure 5.2: LSTM cell with forget, input, and output gates, plus cell state highway.

Chapter 6

Attention, Memory, and Neural Turing Machines

6.1 Attention Mechanisms

Definition 6.1.1: Attention

Given queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$, *Scaled Dot-Product Attention* computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

The scaling factor $1/\sqrt{d_k}$ stabilises gradients.

6.1.1 Multi-Head Attention (Transformer)

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O,$$

where $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

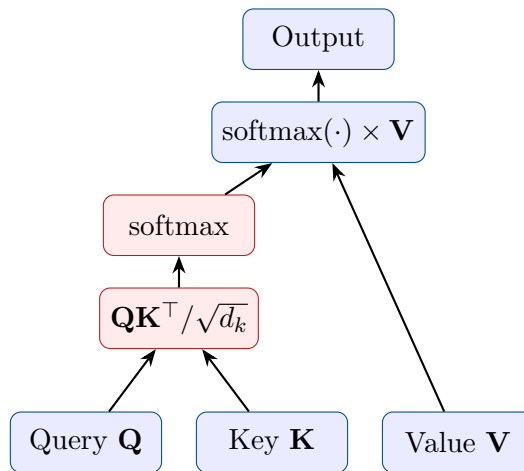


Figure 6.1: Scaled dot-product attention computation graph.

6.2 Memory Networks

Memory Networks (Weston et al., 2015) augment neural networks with an external memory matrix $\mathbf{M} \in \mathbb{R}^{N_m \times d}$. At inference, the model computes soft attention over memory slots:

$$p_i = \text{softmax}(\mathbf{q}^\top \mathbf{m}_i), \quad \mathbf{o} = \sum_i p_i \mathbf{c}_i,$$

where $\mathbf{q}$ is a query, $\mathbf{m}_i$ are input embeddings and $\mathbf{c}_i$ are output embeddings of memory slots. **End-to-End Memory Networks** stack multiple memory hops for iterative reasoning: $\mathbf{u}^{k+1} = \mathbf{u}^k + \mathbf{o}^k$.

6.3 Neural Turing Machines (NTMs)

An NTM (Graves et al., 2014) couples an LSTM controller with an external memory bank $\mathbf{M}_t \in \mathbb{R}^{N \times W}$. Read/write heads produce attention weights $\mathbf{w}_t \in \Delta^{N-1}$:

$$\mathbf{r}_t = \sum_i w_t(i) \mathbf{M}_t(i), \quad (\text{read})$$

$$\mathbf{M}_t(i) = \mathbf{M}_{t-1}(i) [1 - w_t^w(i) e_t] + w_t^w(i) \mathbf{a}_t. \quad (\text{write})$$

Addressing combines:

1. *Content-based*: cosine similarity between key $\mathbf{k}_t$ and rows of $\mathbf{M}_t$.
2. *Location-based*: convolutional shift $\mathbf{w}_t^g = \sum_j s_t(j) \tilde{w}_{(i-j)}$.

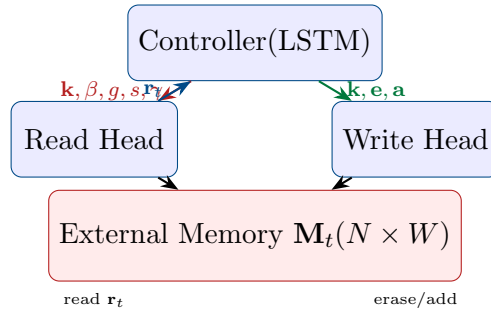


Figure 6.2: Neural Turing Machine: controller interacts with external memory via differentiable read/write heads.

Chapter 7

NLP Applications

7.1 Machine Translation

7.1.1 Encoder–Decoder Architecture

$$\mathbf{h}_t^{\text{enc}} = \text{LSTM}_{\text{enc}}(\mathbf{x}_t, \mathbf{h}_{t-1}^{\text{enc}}), \quad \mathbf{s}_j = \text{LSTM}_{\text{dec}}(\mathbf{s}_{j-1}, \mathbf{c}_j),$$

$$\mathbf{c}_j = \sum_t \alpha_{jt} \mathbf{h}_t^{\text{enc}}, \quad \alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{t'} \exp(e_{jt'})}, \quad e_{jt} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{j-1} + \mathbf{U}_a \mathbf{h}_t).$$

7.1.2 Transformer for Translation

The Transformer (Vaswani et al., 2017) replaces recurrence entirely:

- **Encoder:** N_e layers of Multi-Head Self-Attention + FFN.
- **Decoder:** N_d layers of Masked MH-Self-Attn + Cross-Attn + FFN.
- **Positional Encoding:** $\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$.

7.2 Question Answering (QA)

- **Extractive QA** (e.g. SQuAD): predict start/end span indices in passage.
- **Abstractive QA**: generate a free-form answer (seq2seq).
- **BERT-based QA**: fine-tune pretrained transformer; output heads predict $P(\text{start})$ and $P(\text{end})$.
- **Retrieval-Augmented Generation (RAG)**: retrieve relevant documents then generate answer conditioned on retrieved context.

7.3 Speech Recognition

Automatic Speech Recognition (ASR) maps an acoustic feature sequence $\mathbf{o}_{1:T}$ (e.g. log-mel filterbanks) to a word sequence $w_{1:L}$.

7.3.1 Connectionist Temporal Classification (CTC)

CTC marginalises over all valid alignments π of length T :

$$p(\mathbf{y} \mid \mathbf{x}) = \sum_{\pi: \mathcal{B}(\pi) = \mathbf{y}} \prod_{t=1}^T p(\pi_t \mid \mathbf{x}),$$

where $\mathcal{B}$ collapses repeated tokens and removes blanks. Forward–backward dynamic programming computes this efficiently.

7.3.2 Attention-Based Encoder–Decoder (AED)

$$s_l, c_l = \text{AttDecoder}(s_{l-1}, y_{l-1}, \mathbf{h}_{1:T}), \quad y_l \sim P_\theta(\cdot \mid s_l, c_l).$$

7.3.3 Whisper / wav2vec 2.0

Modern end-to-end models use self-supervised pretraining on large unlabelled audio corpora, followed by supervised fine-tuning, achieving near-human performance on standard benchmarks.

7.4 Syntactic and Semantic Parsing

7.4.1 Syntactic Parsing

Constituency parsing produces a phrase-structure tree; **dependency parsing** labels directed arcs between words. Neural approaches use:

- **BiLSTM + CRF**: contextualised token representations + structured output.
- **Graph Neural Networks (GNNs)**: encode tree structure explicitly.
- **Transformer span-based**: score all spans via self-attention representations.

7.4.2 Semantic Parsing

Maps natural language to a formal meaning representation (e.g. SQL, SPARQL, λ -calculus, AMR). Sequence-to-sequence models with copy mechanisms (pointer networks) handle novel entities. **T5/CodeT5** achieve state-of-the-art on Text-to-SQL tasks.

Chapter 8

GPU Optimisation for Neural Networks

8.1 Hardware Landscape

Modern GPUs (e.g. NVIDIA A100/H100) provide:

- Thousands of CUDA cores for parallel floating-point ops.
- Tensor Cores: hardware-accelerated mixed-precision matrix multiplication.
- High-bandwidth memory (HBM): >2 TB/s bandwidth (H100 SXM).

8.2 Memory Optimisation

- **Mixed Precision (FP16/BF16):** halves memory; use FP32 master weights.
- **Gradient Checkpointing:** recompute activations during backward pass to reduce peak memory by $O(\sqrt{n})$ at cost of $O(1)$ extra forward pass.
- **Activation Offloading:** swap tensors to CPU/NVMe when idle.
- **FlashAttention:** fuses attention operations to avoid $O(L^2)$ memory materialisation; operates in SRAM tiles for $O(L)$ memory.

8.3 Compute Optimisation

- **Data parallelism:** replicate model; aggregate gradients via AllReduce.
- **Model parallelism:** partition layers across GPUs (tensor/pipeline parallel).
- **Fused kernels:** combine elementwise ops (e.g. layer-norm + GELU) to reduce kernel launch overhead.
- **Quantisation:** INT8/INT4 weights reduce memory and increase throughput (e.g. GPTQ, AWQ).
- **Compilation (torch.compile / XLA):** static graph optimisation, operator fusion, and autotuned tile sizes.

8.4 Distributed Training at Scale

The NCCL (NVIDIA Collective Communications Library) implements efficient multi-GPU communication primitives. **ZeRO** (Zero Redundancy Optimizer) partitions optimizer states, gradients, and parameters across GPUs to scale to trillion-parameter models. The total communication volume per step is $O(3\Psi)$ for ZeRO-3 vs. $O(2\Psi)$ for standard data parallelism, where Ψ is model size.

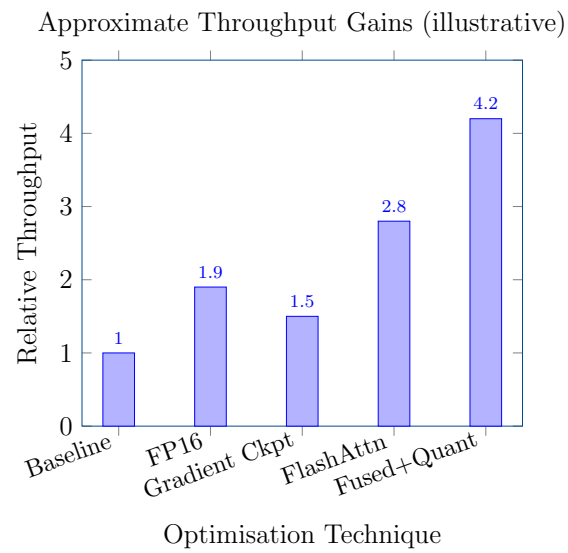


Figure 8.1: Illustrative relative throughput gains from common GPU optimisation techniques applied cumulatively.

Key References

1. C.E. Rasmussen & C.K.I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.
2. D.P. Kingma & M. Welling, “Auto-Encoding Variational Bayes”, ICLR 2014.
3. I.J. Goodfellow et al., “Generative Adversarial Nets”, NeurIPS 2014.
4. S. Hochreiter & J. Schmidhuber, “Long Short-Term Memory”, *Neural Computation*, 1997.
5. A. Vaswani et al., “Attention Is All You Need”, NeurIPS 2017.
6. A. Graves et al., “Neural Turing Machines”, arXiv 2014.
7. Y. Gal & Z. Ghahramani, “Dropout as a Bayesian Approximation”, ICML 2016.
8. T. Dao et al., “FlashAttention”, NeurIPS 2022.
9. S. Rajbhandari et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models”, SC 2020.
10. A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision” (Whisper), ICML 2023.